Technology Selection

Project: AI-Assisted Travel Planning Web Application

1. Purpose and Architectural Alignment

This review identifies suitable technologies for the components in the TripMind UML Component Diagram and compares three Python libraries for interacting with LLM services. The recommendations describe the proposed implementation, rather than completed functionality.

The existing prototype uses React and Vite with simulated itinerary data. Flask, live LLM integration and server-side validation remain to be implemented. SQLite is retained here to match the proposed UML architecture; it is not an existing database.

2. Technologies for Each UML Component

Web Front-end

React, Vite, JavaScript/JSX, CSS and the Fetch API. Retain the existing interface for destination, dates, budget, travellers and interests. React displays daily itineraries and manages loading and error states; Fetch sends JSON requests to the backend.

API Layer (Flask Routes)

Python and Flask. Expose a generation endpoint, receive user preferences and return structured JSON. The diagram names /api/generate, with /api/validate and /api/attractions optional. The frontend documentation suggests /api/trips/generate; agree one route before integration. Validation can run inside generation without a separate endpoint.

Travel Business Logic

Python, Pydantic, datetime and decimal. Validate input fields, process preferences, retrieve relevant attraction records and apply budget and time rules. Explicit Python checks should verify date coverage, activity overlaps and recalculated costs. Pydantic supports data schemas and custom validation [1].

LLM Interaction Component

A selected LLM library plus Pydantic. Build prompts from preferences and constraints, call the external service, parse responses and handle failures. Define a shared itinerary schema for daily activities, estimated costs and uncertainty notes. Keep API credentials in backend environment configuration.

Data Access Layer

Python sqlite3. Encapsulate queries for attractions, cities and categories. The built-in library is sufficient for a small dataset. SQLAlchemy is an alternative if more complex database relationships justify an object-relational mapping layer.

Database (SQLite)

SQLite. Store a small, curated attraction dataset with source links and verification dates. User history remains optional. Existing browser localStorage saving is separate from this proposed server-side database.

External LLM Services

OpenAI, Anthropic or Google. These are model providers, whereas the Python libraries are integration tools. Select one service for the initial application; the diagram does not require all providers to be connected.

3. Role and Limits of Pydantic

Pydantic is a supporting library, not a separate UML component. It can validate both user requests and model outputs. Correct structure does not establish factual accuracy: prices, opening hours and travel feasibility still require evidence or explicit uncertainty labels.

4. Comparison of Python LLM Libraries

OpenAI Python SDK (openai)

- **Strengths:** Direct access to OpenAI services and structured-output helpers that parse responses into Pydantic models [2]. A small integration is straightforward to inspect and explain.
- **Limitations:** Primarily provider-specific. Other services may require separate integrations, and structured-output availability depends on the chosen model.
- **Suitability:** The preferred starting point if OpenAI is selected for the MVP.

LiteLLM (litellm)

- **Strengths:** A common Python interface across multiple providers, with normalized responses and errors. Useful for changing providers while retaining similar application code [3].
- **Limitations:** Adds an abstraction layer. Provider capabilities and supported schema options still differ; compatibility must be checked for each model [4].
- **Suitability:** A strong candidate when cross-provider experiments are central to the dissertation.

LangChain (langchain and provider integration packages)

- **Strengths:** Standard model interfaces, structured-output support and facilities for composing model workflows [5].
- **Limitations:** More concepts and dependencies to manage. Its broader workflow features offer limited benefit for a simple request, generation and validation pipeline.
- **Suitability:** Consider if requirements develop into more complex workflows; not necessary for the initial MVP.

5. Preliminary Recommendation

Retain React and Vite, implement the backend with Flask, use Pydantic for shared data schemas and use sqlite3 with SQLite for the diagram's data components. Begin with a direct provider SDK once the provider is chosen; use LiteLLM instead if provider comparison is a core requirement. Isolate model calls in one backend module so the integration can be replaced.

Keep weather, maps, booking, accounts and complex multi-agent workflows outside the initial scope. Optional routes and user history should only be implemented if needed. Update the UML if the final storage or component responsibilities change.

6. Proposed Evaluation

This comparison is based on documentation and project fit; no library benchmark has been run. Where supported, compare libraries using the same model, prompt, schema and travel inputs, with equivalent settings and repeated trials.

- Record setup effort, structured-output success, error handling, latency, token usage and ease of switching providers. Record versions and retry settings.
- Evaluate itinerary relevance, budget consistency and date coverage separately from integration-library suitability. Model and prompt differences can affect these outcomes.

Use pytest and Flask's test client for validation and API failure cases [6]. Retain prompts, model identifiers, raw outputs and evaluation results in JSONL or CSV for reproducible project evidence.

7. References

[1] Pydantic documentation. https://docs.pydantic.dev/latest/

[2] OpenAI: Structured model outputs. https://developers.openai.com/api/docs/guides/structured-outputs

[3] LiteLLM: Python SDK. https://docs.litellm.ai/docs/python_sdk

[4] LiteLLM: Structured outputs. https://docs.litellm.ai/docs/completion/json_mode

[5] LangChain: Models. https://docs.langchain.com/oss/python/langchain/models

[6] Flask: Testing Flask applications. https://flask.palletsprojects.com/en/stable/testing/

Documentation reviewed: 14 October 2026. Architecture reference: TripMind UML Component Diagram.